核桃编程

L12-4　模块1：模拟法的应用
模块2：进制相关的模拟

1. 单词卡

long long	base
长整型	进制

2. 知识探索

（1）模拟法的应用场景

模拟法通常用于解决那些可以通过逐步模拟过程来找到解决方案的问题。

（2）猜数字

1. 人物 a 设定一个数字 x；
2. 让人物 b 来猜数；
3. a 根据 b 的猜测给出提示，大了/小了/猜对了；
4. 重复 2，3 步，直到猜对后结束。

```
int a, x;
cin >> x;
cin >> a;
while (a != x)
{
    if (a < x)
    {
        cout << "小了!再试一次:" << endl;
    }
    if (a > x)
    {
        cout << "大了!再试一次:" << endl;
    }
    cin >> a;
}
cout << "恭喜你,猜中了!正确答案是" << x << "!";
return 0;
```

使用while循环
模拟猜数过程

（3）long long 类型

long long 为长整型，数据范围为 - 2^63 ~ 2^63 -1

大约为： - 9 * 10^18 ~ 9 * 10^18

用法和int类似：

```
long long a;
long long b[105];
long long f(long long x, long long y)
{
    return x + y;
}
```

（4）一题多解

问题描述：

一段旋律由7个音符组成，每个音符用数字1到7编号，这些音符可以是升序的（从1到7），也可以是降序的（从7到1），或是混合的，请你根据音符的顺序，判断旋律类型。

方法一：

1. 使用数组存储音符；
2. 根据题目描述判断音符；
3. 使用两个函数分别判断音符是否为升序或降序；
4. 若都不符，则说明是混合

```
bool Is_up() // 判断是否为升序(不存在相等情况)
{
    for (int i = 2; i <= 7; i++)
        if (a[i] < a[i - 1])
            return false;
    return true;
}
```

```
bool Is_down() // 判断是否为降序(不存在相等情况)
{
    for (int i = 2; i <= 7; i++)
        if (a[i] > a[i - 1])
            return false;
    return true;
}
```

```
if (Is_up())
    cout << "升序";
else if (Is_down())
    cout << "降序";
else // 否则是混合的
    cout << "混合";
```

方法二：

1. 使用7个变量存储音符；
2. 根据题目描述判断音符；
3. 使用 if ... else if ... else 语句依次判断音符是否为升序、降序或是混合。

```
int a, b, c, d, e, f, g;
cin >> a >> b >> c >> d >> e >> f >> g;
if (a<b && b<c && c<d && d<e && e<f && f<g)
    cout << "升序";
else if (a>b && b>c && c>d && d>e && e>f && f>g)
    cout << "降序";
else
    cout << "混合";
```

方法三：

1. 循环输入每个音符，将它们转换为一个7位整数；
2. 根据题目描述判断音符；
3. 升序和降序对应的7位整数都是唯一的；
4. 依次判断音符是否为升序、降序或是混合。

```
int sum = 0, a; // sum是转化后的数
for (int i = 1; i <= 7; i++)
{
    cin >> a;
    sum = sum * 10 + a; // 将7个数字转换成一个数
}
if (sum == 1234567)
    cout << "升序";
else if (sum == 7654321)
    cout << "降序";
else
    cout << "混合";
```

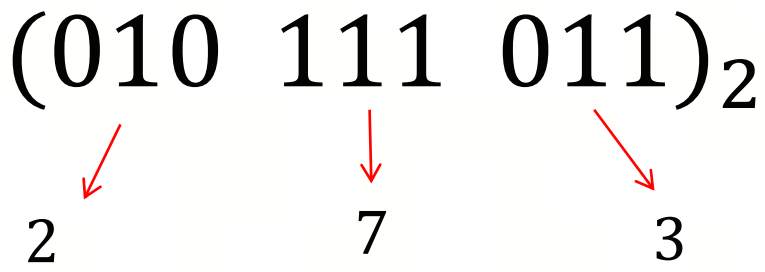
（5）八进制和二进制的转换

八进制中的1位，对应二进制中的3位：

八进制	二进制
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

二进制转八进制：

先从右往左把二进制每三位拆分开，不足三位高位补零，然后依次转换为八进制。



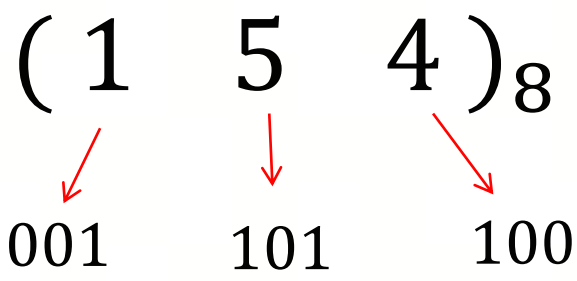

```
string s;
cin >> s;
while (s.length() % 3)
{
    s = '0' + s;
}
for (int i = 2; i < s.length(); i += 3)
{
    int x = 0;
    x += (s[i-2]- '0') * 4;
    x += (s[i-1]- '0') * 2;
    x += s[i]- '0';
    cout << x;
}
```

若长度不是3的倍数，最高位补0

每3位转换为8进制数

八进制转二进制：

依次把八进制的每一位都转换为三位二进制数，然后将最终二进制数中的前导**0**去掉即可。




```
string s;
cin >> s;
if (s[0] == '1')    cout << "1";
if (s[0] == '2')    cout << "10";
if (s[0] == '3')    cout << "11";
if (s[0] == '4')    cout << "100";
if (s[0] == '5')    cout << "101";
if (s[0] == '6')    cout << "110";
if (s[0] == '7')    cout << "111";
for (int i = 1; i < s.length(); i++)
{
    if (s[i] == '0')    cout << "000";
    if (s[i] == '1')    cout << "001";
    if (s[i] == '2')    cout << "010";
    if (s[i] == '3')    cout << "011";
    if (s[i] == '4')    cout << "100";
    if (s[i] == '5')    cout << "101";
    if (s[i] == '6')    cout << "110";
    if (s[i] == '7')    cout << "111";
}
```

单独处理s[0]的情况